

Web 网盘项目

peanutixx

2026 年 4 月 17 日

Contents

1 第一期：实现基本功能	2
1.1 数据库表的设计	2
1.2 注册	3
1.3 登录	4
1.4 获取当前用户信息	5
1.5 文件列表查询	5
1.6 上传文件	6
1.7 下载文件	7
1.8 小结	8
2 Docker	8
2.1 简介	8
2.2 核心概念	8
2.3 安装	9
2.4 Docker 命令	11
2.5 Docker 实验	12
3 第二期：阿里云对象存储 OSS	13
3.1 快速了解 OSS	13
3.2 控制台操作	14
3.3 安装 OSS C++ SDK	14
3.4 示例：上传文件	15
3.5 小结	17
4 第三期：消息队列 RabbitMQ	17
4.1 引入消息队列带来的好处	17
4.2 RabbitMQ 简介	17
4.3 使用 Docker 安装 RabbitMQ	18
4.4 RabbitMQ 的架构和核心概念	18
4.5 控制台操作	19
4.6 安装 SimpleAmqpClient	19
4.7 示例：生产者发送消息	20
4.8 示例：消费者消费消息	20
5 第四期：微服务架构	21
5.1 单体应用 v.s. 微服务应用	22
5.2 Protobuf	23
5.3 sRPC	26

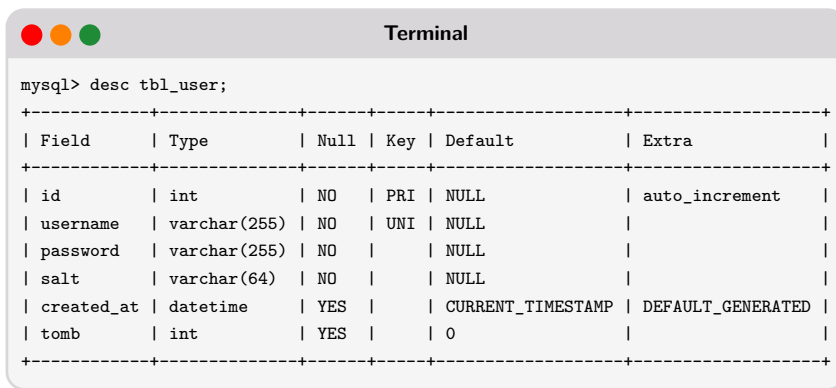
6 第五期：注册中心 Consul	29
6.1 简介	30
6.2 使用 Docker 安装 Consul	30
6.3 安装 ppconsul	31
6.4 使用教程	31

1 第一期：实现基本功能

这是一个教学用的项目，总共分为 5 期，每一期我们都会逐步加入一些新的功能。

1.1 数据库表的设计

这个项目只会涉及到两张表，分别为 `tbl_user` 和 `tbl_file`。`tbl_user` 表的结构如下：



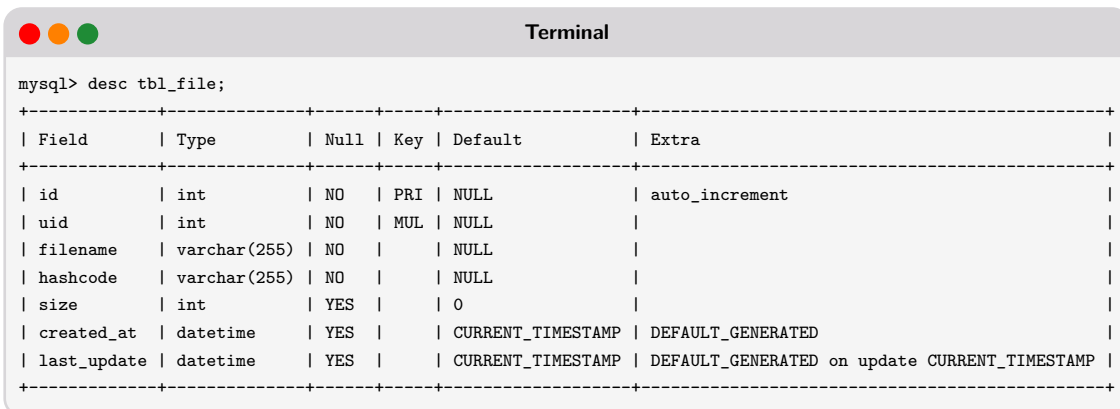
```
mysql> desc tbl_user;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
username	varchar(255)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
salt	varchar(64)	NO		NULL	
created_at	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
tomb	int	YES		0	

`tbl_user` 表的建表语句：

```
CREATE TABLE tbl_user(
  id int PRIMARY KEY AUTO_INCREMENT,
  username varchar(255) NOT NULL UNIQUE,
  password varchar(255) NOT NULL,
  salt varchar(64) NOT NULL,
  created_at datetime DEFAULT CURRENT_TIMESTAMP(),
  tomb int DEFAULT 0
);
```

`tbl_file` 表的结构如下：



```
mysql> desc tbl_file;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
uid	int	NO	MUL	NULL	
filename	varchar(255)	NO		NULL	
hashcode	varchar(255)	NO		NULL	
size	int	YES		0	
created_at	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
last_update	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TIMESTAMP

tbl_user 表的建表语句:

```
CREATE TABLE tbl_file(  
    id int PRIMARY KEY AUTO_INCREMENT,  
    uid int NOT NULL,  
    filename varchar(255) NOT NULL,  
    hashcode varchar(255) NOT NULL,  
    size int DEFAULT 0,  
    created_at datetime DEFAULT CURRENT_TIMESTAMP(),  
    last_update datetime DEFAULT CURRENT_TIMESTAMP() ON UPDATE CURRENT_TIMESTAMP(),  
    UNIQUE KEY (uid, filename)  
);
```

1.2 注册

1. API 端点¹: POST /api/v1/auth/register。

2. 请求:

```
POST /api/v1/auth/register HTTP/1.1  
Content-Type: application/json    // 省略其它头部字段  
  
{  
    "username": "peanutixx",  
    "password": "1234",  
    "confirm": "1234"  
}
```

3. 响应:

成功返回响应状态码 201, 并返回下面格式的 JSON 数据。

```
HTTP/1.1 201 Created  
Content-Type: application/json    // 省略其它头部字段  
  
{  
    "status": "success",  
    "message": "注册成功",  
    "data": {  
        "userId": 1,  
        "username": "peanutixx",  
    }  
}
```

失败, 返回下面格式的 JSON 数据, 并设置对应的响应状态码:

```
{  
    "status": "error",  
    "message": <具体的出错原因>  
}
```

¹API 端点: 指客户端与 API 进行交互的具体访问地址 (URL)。通俗地将, API 就像政府办公大楼, API 端点就是具体的办公窗口。

原因	响应状态码	message
请求类型不是 <code>application/json</code>	400	"请求格式有误"
用户名或密码为空	400	"用户名和密码不能为空"
密码和确认密码不一致	400	"两次输入的密码不一致"
插入记录失败	409	"用户名已存在"

1.3 登录

1. API 端点: `POST /api/v1/auth/login`。

2. 请求:

```
POST /api/v1/auth/login HTTP/1.1
Content-Type: application/json    // 省略其它头部字段

{
  "username": "peanutixx",
  "password": "1234"
}
```

3. 响应:

成功返回响应状态码 200, 并返回下面格式的 JSON 数据。

```
HTTP/1.1 200 OK
Content-Type: application/json    // 省略其它头部字段

{
  "status": "success",
  "message": "登录成功",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "tokenType": "Bearer",
    "user": {
      "userId": 1,
      "username": "peanutixx",
    }
  }
}
```

失败, 返回下面格式的 JSON 数据, 并设置对应的响应状态码:

```
{
  "status": "error",
  "message": "<具体的出错原因>"
}
```

原因	响应状态码	message
请求类型不是 <code>application/json</code>	400	“请求格式有误”
用户名或密码为空	400	“用户名和密码不能为空”
密码不对	401	“用户名或密码错误”
执行 SQL 语句失败	500	“内部服务器错误”
SQL 返回空结果集	401	“用户名或密码错误”

1.4 获取当前用户信息

1. API 端点: `GET /api/v1/user/me`。

2. 请求:

```
GET /api/v1/user/me HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

// 省略其它头部字段

3. 响应:

成功返回响应状态码 200, 并返回下面格式的 JSON 数据。

```
HTTP/1.1 200 OK
Content-Type: application/json
```

// 省略其它头部字段

```
{
  "status": "success",
  "message": "获取个人信息成功",
  "data": {
    "userId": 1,
    "username": "peanutixx",
    "createdAt": "2026-04-02 12:04:01"
  }
}
```

失败, 返回下面格式的 JSON 数据, 并设置对应的响应状态码:

```
{
  "status": "error",
  "message": "<具体的出错原因>"
}
```

原因	响应状态码	message
没有令牌, 或者令牌的类型不是 <code>Bearer</code> ²	401	“无效的访问令牌”
令牌验证失败	401	“无效的访问令牌”

1.5 文件列表查询

1. API 端点: `GET /api/v1/files`。

² 承载令牌: 最常见的令牌使用方式, “谁持有谁就是令牌的主人”。客户端不需要证明自己真实的身份, 令牌本身就是凭证。

2. 请求:

```
GET /api/v1/files HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... // 省略其它头部字段
```

3. 响应:

成功返回响应状态码 200, 并返回下面格式的 JSON 数据。

```
HTTP/1.1 200 OK
Content-Type: application/json // 省略其它头部字段
```

```
{
  "status": "success",
  "message": "获取文件列表成功",
  "data": {
    "files": [
      {
        "fileId": 6,
        "filename": "AlgorithmicPuzzles.pdf",
        "size": 1647220,
        "createdAt": "2026-04-02 21:06:29",
        "updatedAt": "2026-04-02 21:06:29"
      },
      {
        "fileId": 7,
        "filename": "images.jpg",
        "size": 5579,
        "createdAt": "2026-04-02 21:07:33",
        "updatedAt": "2026-04-02 21:07:33"
      }
    ]
  }
}
```

失败, 返回下面格式的 JSON 数据, 并设置对应的响应状态码:

```
{
  "status": "error",
  "message": <具体的出错原因>
}
```

原因	响应状态码	message
没有令牌, 令牌类型不是 Bearer , 令牌校验失败	401	"无效的访问令牌"
SQL 语句执行失败	500	"内部服务器错误"

1.6 上传文件

1. API 端点: POST /api/v1/files。

2. 请求:

```
POST /api/v1/files HTTP/1.1
Content-Type: multipart/form-data; boundary=...
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... // 省略其它头部字段

// 请求体
```

3. 响应:
成功返回响应状态码 200, 并返回下面格式的 JSON 数据。

```
HTTP/1.1 201 Created
Content-Type: application/json // 省略其它头部字段

{
  "status": "success",
  "message": "上传成功",
  "data": {
    "fileId": 22,
    "filename": "a.txt"
  }
}
```

失败, 返回下面格式的 JSON 数据, 并设置对应的响应状态码:

```
{
  "status": "error",
  "message": <具体的出错原因>
}
```

原因	响应状态码	message
没有令牌, 令牌类型不是 Bearer, 令牌校验失败	401	"无效的访问令牌"
请求体类型不为 multipart/form-data	400	"请求格式有误"
SQL 语句执行失败	500	"内部服务器错误"

1.7 下载文件

1. API 端点: GET /api/v1/file/{id}。

2. 请求:

```
GET /api/v1/file/22 HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... // 省略其它头部字段
```

3. 响应:
成功返回响应状态码 200, 并返回下面格式的 JSON 数据。

```
HTTP/1.1 200 OK
Content-Disposition: attachment; filename=a.txt // a.txt 要替换成具体的文件名, 省略其它头部字段
```

// 文件内容...

失败，返回下面格式的 JSON 数据，并设置对应的响应状态码：

```
{
  "status": "error",
  "message": <具体的出错原因>
}
```

原因	响应状态码	message
没有令牌，令牌类型不是 Bearer ，令牌校验失败	401	“无效的访问令牌”
请求的文件不存在	404	“文件不存在”
SQL 语句执行失败	500	“内部服务器错误”

1.8 小结

完成第一期功能之后，相信大家已经熟悉了后端 API 端点的开发流程了。后端开发的首要原则就是：不要相信任何客户端传过来的数据！因此，后端绝大多数代码都是在做边界检查、错误处理，核心代码的占比可能不足 1/3。这样做的目的是：保证系统的健壮性（Robust）³。

2 Docker

2.1 简介

Docker 是一种轻量级的虚拟化技术，它让应用程序及其依赖环境可以被打包成一个标准化的单元（镜像），在任何地方都能一致地运行。

如果用一个生活中的例子类比：Docker 之于软件，就像集装箱之于货物。在集装箱发明之前，货物的运输是一件麻烦的事情：不同的货物需要不同的包装、不同的装卸方式，换一种运输工具就要重新装卸。集装箱的出现改变了这一切：无论里面装的是什么，集装箱的外形是标准的，可以用同样的方式装卸、堆放和运输。Docker 做的事情类似：无论你的应用是用 Python、C++、Java 还是其他语言写的，无论它需要什么样的依赖库和环境，一旦被打包成 Docker 镜像，就可以用同样的方式在任何支持 Docker 的机器上运行。

Docker 的口号是：“build once, run everywhere”，一次构建，处处运行。Docker 是一项革命性的技术，它彻底改变了开发和交付的方式。

2.2 核心概念

Docker 有三个最核心的概念：

- 镜像（Image）

Docker 镜像是一个特殊的文件系统，除了提供容器运行时所需的程序、库、资源、配置等文件外，还包含了一些为运行时准备的一些配置参数。镜像不包含任何动态数据，其内容在构建之后也不会被改变。

- 容器（Container）

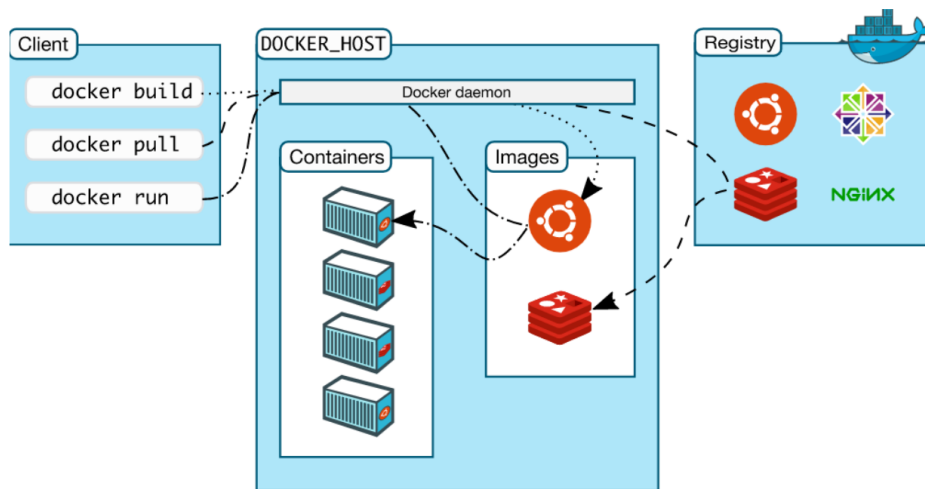
镜像（Image）和容器（Container）的关系，就像是程序和进程一样，镜像是静态的定义，容器是镜像运行时的实体。容器可以被创建、启动、停止、删除、暂停等。

- 仓库（Repository）

镜像构建完成后，可以很容易的在当前宿主机上运行，但是，如果需要在其它服务器上使用这个镜像，我们就需要一个集中的存储、分发镜像的服务，Docker Registry 就是这样的服务。Docker Hub 是 Docker 官方的 Registry，里面存放了很多镜像的集合，镜像的集合我们称之为 Repository，比如：library/ubuntu。

³健壮性：通俗地讲，就是系统的“抗打击能力”或“皮实程度”。一个健壮的程序，不是“不会出错”，而是在出错时不会崩溃或产生灾难性后果。

理解了这三个概念，对理解 Docker 至关重要。



2.3 安装

本节将介绍如何在 Ubuntu 系统上安装 Docker，并配置国内镜像加速。在开始安装之前，我们需要确认系统版本是否满足要求，并清理可能存在的旧版本。

1. 系统要求

目前最新的 Docker 引擎，支持下面 64 位版本的 Ubuntu：

- Ubuntu Resolute 26.04 (LTS)
- Ubuntu Questing 25.10
- Ubuntu Noble 24.04 (LTS)
- Ubuntu Jammy 22.04 (LTS)

执行命令 `cat /etc/*release` 查看你的 Linux 系统版本信息：

```
Terminal

peanut@wd:~$ cat /etc/*release
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=22.04
DISTRIB_CODENAME=jammy
DISTRIB_DESCRIPTION="Ubuntu 22.04.5 LTS"
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
```

2. 卸载旧版本，如果有的话

```
sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc
```

```
podman-docker containerd runc | cut -f1)
```

3. 设置 *Docker* 的 *apt* 仓库

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
# 由于众所周知的原因，下面这条命令可能需要多执行几次才能成功
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

4. 安装 *Docker*

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

5. 修改镜像源

将下面内容添加到 `/etc/docker/daemon.json` 中，如果文件不存在，则自己创建。

```
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io/",
    "https://dockerpull.com",
    "https://docker-0.unsee.tech",
    "https://docker-cf.registry.cyou",
    "https://docker.1panel.live"
  ]
}
```

6. 重启 *Docker* 服务，并设置开机自启动

```
sudo systemctl daemon-reload
sudo systemctl restart docker
sudo systemctl enable docker
```

7. 将当前用户加入到 *docker* 组

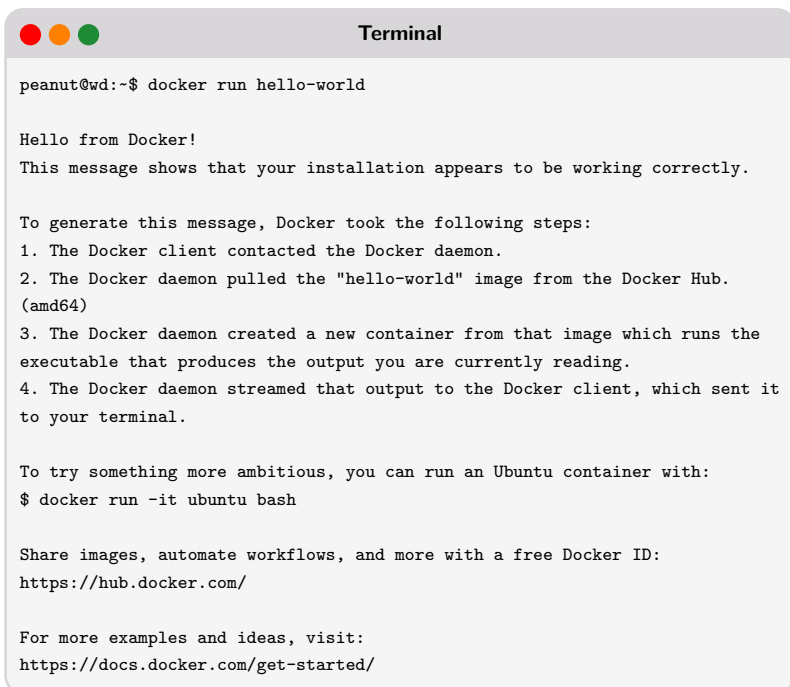
Docker 客户端得使用 `/var/run/docker.sock` 这个本地套接字文件与 *Docker* 引擎通信，而这个文件默认情况下只有 `root` 和 *docker* 组的用户才能访问。

```
# 如果系统没有 docker 组，使用下面命令创建
# sudo groupadd docker
sudo usermod -aG docker $USER
```

退出当前终端并重新登录。

8. 验证安装是否成功

执行 `docker run hello-world`，出现下面界面，说明安装成功。

A terminal window titled "Terminal" with a macOS-style title bar (red, yellow, green buttons). The prompt is "peanut@wd:~\$". The command "docker run hello-world" has been executed. The output is a multi-line message from Docker, including a greeting, a confirmation of successful installation, a list of steps taken by Docker (pulling the image, creating a container, streaming output), and links to Docker Hub and documentation.

```
peanut@wd:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

详细信息见：[Install Docker Engine](#)

2.4 Docker 命令

帮助手册

Docker CLI 有一个非常好用的帮助手册：`docker <command> --help`。

镜像相关命令

命令	解释
<code>docker pull</code>	从仓库拉取（下载）镜像
<code>docker push</code>	推送（上传）一个镜像到仓库
<code>docker images</code>	查看镜像
<code>docker rmi</code>	删除镜像
<code>docker save</code>	将镜像导出（保存）为 <code>tar</code> 包
<code>docker load</code>	从 <code>tar</code> 包加载一个镜像

容器相关命令

命令	解释
<code>docker run</code> -d: 后台启动 --name: 指定容器名 -p: 端口映射 -v: 目录挂载 --network: 加入指定的网络	依据镜像创建并启动一个新容器
<code>docker ps</code> -a: 查看所有容器（默认只查看正在运行的） -q: 只显示容器的 ID	查看容器
<code>docker stop</code>	停止一个或多个容器
<code>docker start</code>	启动一个或多个已停止的容器
<code>docker restart</code>	重启一个或多个容器
<code>docker rm</code> -f: 强制删除正在运行的容器	删除一个或多个容器
<code>docker exec</code> -i: 交互式（即使在后台运行，也能接收终端输入的字符） -t: 分配一个伪终端	在一个正在运行的容器中执行一条命令
<code>docker logs</code>	查看容器运行的日志
<code>docker inspect</code>	查看容器的详细信息
<code>docker commit</code>	将容器的当前状态保存为镜像

接下来，我们会通过一些具体的实验，让大家练习上面罗列的命令。

2.5 Docker 实验

实验 1：流程

1. 启动一个 Nginx 容器
2. 修改 Nginx 的首页（`/usr/share/nginx/html/index.html`）
3. 将修改后的容器保存为镜像
4. 将镜像保存为 tar 包
5. 停止并删除 Nginx 容器
6. 从 tar 包导入镜像
7. 从新导入的镜像，启动 Nginx 容器
8. 查看 Nginx 的首页，看是否有被修改

实验 2：挂载

`docker run -v` 选项用于在容器和宿主机之间实现绑定挂载（Bind Mount）和卷挂载（Volume Mount），实现数据持久化和文件共享。

1. 绑定挂载

将宿主机目录映射到容器内：

```
# 绝对路径
docker run -v /app/nginx/www:/usr/share/nginx/html nginx
# 相对路径
docker run -v ./www:/usr/share/nginx/html nginx
```

2. 卷挂载

卷是 Docker 管理的特殊目录，用于在容器之间共享和持久化数据。与容器内的普通文件系统不同，卷的生命周期独立于容器。

```
# 创建卷
docker volume create myvolume
# 卷挂载
docker run -v myvolume:/usr/share/nginx/html nginx
```

实验 3：自定义网络

Docker 为每个容器都分配了唯一的 ip 地址，容器使用 ip 地址和端口，可以相互访问。

Docker 启动时会自动创建 `docker0` 虚拟网桥（默认的 `bridge` 网络），所有未指定网络的容器都会连接到这个网桥上。但在生产环境下，我们不推荐使用默认的 `bridge` 网络（`docker0`），而是推荐使用自定义网络。因为默认的 `bridge` 网络只能用 IP 地址通信，不支持容器名 DNS 解析。

```
# 创建网络
docker network create mynetwork
# 查看网络详情
docker network inspect mynetwork
# 启动容器并连接到自定义网络
docker run -d --name app1 --network mynetwork nginx
```

3 第二期：阿里云对象存储 OSS

在企业开发中，容灾⁴备份⁵是至关重要的。因此，我们需要一套成熟的方案来解决这个问题。遗憾的是，企业自建解决方案，基本上不可能的。原因有两个：1) 成本太高；2) 无论是数据同步还是错误恢复，实现难度都非常大。

这种情况下，企业往往会使用现有的成熟的云存储方案。使用云存储方案之后，数据丢失的问题基本上就不会发生了，而且也减少了开发和运维的工作量。在我们这个项目中，我们使用的云产品是阿里云对象存储 OSS。

3.1 快速了解 OSS

请查看阿里云官方网站：[什么是对象存储 OSS](#)。

核心概念：

想要顺利使用 OSS，我们必须先了解下面几个基本概念：

- 存储空间（Bucket）

存储空间是用户用于存储对象（Object）的容器，所有的对象都必须隶属于某个存储空间。存储空间具有各种配置属性，包括地域、访问权限、存储类型等。用户可以根据实际需求，创建不同类型的存储空间来存储不同的数据。

⁴ 容灾：容灾的目的是当主系统所在地发生大范围灾难时（比如地震等），能够快速在另一个地方把整个业务系统重新搭建起来。

⁵ 备份：简单来说，就是为数据创建一个独立的、可恢复的副本。

- 对象 (Object)

对象是 OSS 存储数据的基本单元，也被称为 OSS 的文件。和传统的文件系统不同，对象没有文件目录层级结构的关系。对象由元数据 (Object Meta)、用户数据 (Data) 和文件名 (Key) 组成，并且由存储空间内部唯一的 Key 来标识。对象元数据是一组键值对，表示了对象的一些属性，例如文件类型、编码方式等信息，同时用户也可以在元数据中存储一些自定义的信息。

- 对象名 (ObjectKey)

有时也叫 ObjectName，是对象所在存储空间 (Bucket) 的完整名称，包含完整路径和后缀名，如：abc/efg/123.jpg。

- Object 类型 — Object 有三种类型：

- **Normal**: 通过简单上传生成的 Object。上传结束之后内容是固定的，只能读取，不能修改。如果 Object 内容发生了改变，只能重新上传同名的 Object 来覆盖之前的内容。简单上传适用于上传小于 5 GB 的单个文件、一次 HTTP 请求交互即可完成上传的场景。
- **Multipart**: 通过分片上传生成的 Object。上传结束之后内容是固定的，只能读取，不能修改。如果 Object 内容发生了改变，只能重新上传同名的 Object 来覆盖之前的内容。分片上传适用于大文件加速上传、网络环境较差、文件大小不确定的场景。
- **Appendable**: 通过追加上传生成的 Object。顾名思义，Appendable 对象是可以追加数据的。追加上传适用于视频监控、视频直播等领域生成的实时视频流场景。

重要：OSS 不支持不同类型的 Object 相互转换。

- Region (地域)

Region 表示 OSS 的数据中心所在物理位置。创建 Bucket 时需指定 Region。Bucket 创建成功后，不能更改 Region。该 Bucket 下所有的 Object 都存储在 Region 对应的数据中心。

- Endpoint (访问域名)

Endpoint 表示 OSS 对外服务的访问域名。OSS 以 HTTP RESTful API 的形式对外提供服务，访问不同的 Region，需要不同的域名。内网和外网访问同一个 Region 所需要的 Endpoint 也是不同的。

详细内容请参见：[各个 Region 对应的 Endpoint](#)。

- AccessKey (访问密钥)

AccessKey 简称 AK，指的是访问身份验证中用到的 AccessKey ID 和 AccessKey Secret。OSS 通过使用 AccessKey ID 和 AccessKey Secret 对称加密的方法来验证某个请求的发送者身份。AccessKey ID 用于标识用户；AccessKey Secret 是用户用于加密签名字符串和 OSS 用来验证签名字符串的密钥，必须保密。

详细内容请参见：[创建 AccessKey](#)。

3.2 控制台操作

详细内容请参见：[控制台快速入门](#)

3.3 安装 OSS C++ SDK

安装依赖库

```
sudo apt install libssl-dev
```

```
sudo apt install libcurl4-openssl-dev
```

解压缩 C++ SDK 安装包

```
tar xzvf aliyun-oss-cpp-sdk-1.10.0.tar.gz
```

```
cd aliyun-oss-cpp-sdk-1.10.0
```

安装 C++ SDK

```
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig
```

安装成功后，`/usr/local/lib/`目录下会多一个静态库 `libalibabacloud-oss-cpp-sdk.a`；`/usr/local/include/`目录下会多一个文件夹 `alibabacloud`，里面放的是头文件。

编译链接选项

重要： OSS C++ SDK 默认关闭了 `rtti` 属性。因此使用 `g++` 编译链接时，请添加 `-fno-rtti -lalibabacloud-oss-cpp-sdk -lcurl -lcrypto -lpthread`。

详细内容请参见：[安装 C++ SDK](#)。

3.4 示例：上传文件

在 OSS 中，操作的基本数据单元是文件（Object）。OSS C++ SDK 提供了丰富的文件上传方式，在我们的项目中，使用简单上传即可。简单上传：包括从内存上传和从本地上传，最大不能超过 5GB。

1. 从内存上传

```
oss_upload1.cc
1 #include <alibabacloud/oss/OssClient.h>
2
3 using namespace std;
4 using namespace AlibabaCloud::OSS;
5
6 int main(void)
7 {
8     // 1. 初始化网络等资源
9     InitializeSdk();
10    // 2. 设置 OSS 账号信息，创建 OssClient
11    string endpoint = "oss-cn-wuhan-lr.aliyuncs.com";
12    string accessKeyId = "LTAI5tHQTUfRRD7DTnRcC9DZ";
13    string accessKeySecret = "BCm5w87LsnCHBL0w6MXwZAnJYYOXoN";
14    string region = "cn-wuhan";
15    ClientConfiguration conf;
16    OssClient client(endpoint, accessKeyId, accessKeySecret, conf);
17    client.SetRegion(region);
18    // 3. 上传文件
19    string bucketName = "peanutixx-oss-demo";
20    string objectName = "dir/demo1.txt";
21    string content = "Hello AlibabaCloud OSS";
22    shared_ptr<iostream> stream = make_shared<stringstream>(move(content));
23    PutObjectRequest request(bucketName, objectName, stream);
24    auto outcome = client.PutObject(request);
25    // 4. 错误处理
26    if (!outcome.isSuccess()) {
27        cout << "PutObject FAILED"
28             << ", code:" << outcome.error().Code()
```

```

29         << " , message:" << outcome.error().Message()
30         << " , requestId:" << outcome.error().RequestId() << endl;
31     exit(1);
32 }
33 // 5. 释放网络等资源。
34 ShutdownSdk();
35 return 0;
36 }

```

2. 从本地上传

```

oss_upload2.cc
1 #include <alibabacloud/oss/OssClient.h>
2
3 using namespace std;
4 using namespace AlibabaCloud::OSS;
5
6 int main(void)
7 {
8     // 1. 初始化网络等资源
9     InitializeSdk();
10    // 2. 设置 OSS 账号信息, 创建 OssClient 对象
11    string endpoint = "oss-cn-wuhan-lr.aliyuncs.com";
12    string accessKeyId = "LTAI5tHQTUfRRD7DTnRcC9DZ";
13    string accessKeySecret = "BCm5w87LsnCHBL0w6MXwZAnJYY0XoN";
14    string region = "cn-wuhan";
15    ClientConfiguration conf;
16    OssClient client(endpoint, accessKeyId, accessKeySecret, conf);
17    client.SetRegion(region);
18    // 3. 上传文件
19    string bucketName = "peanutixx-oss-demo";
20    string objectName = "dir/demo2.txt";
21    auto outcome = client.PutObject(bucketName, objectName, "a.txt");
22    // 4. 处理错误
23    if (!outcome.isSuccess()) {
24        cout << "PutObject FAILED"
25             << " , code:" << outcome.error().Code()
26             << " , message:" << outcome.error().Message()
27             << " , requestId:" << outcome.error().RequestId() << endl;
28        exit(1);
29    }
30    // 5. 释放网络等资源
31    ShutdownSdk();
32    return 0;
33 }

```

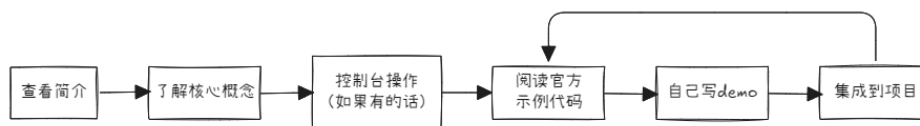
注意事项

在整个程序的生命周期中, `InitializeSdk()` 和 `ShutdownSdk()` 应该各只执行一次! 使用 SDK 之前, 执行 `InitializeSdk()`; SDK 使用完毕之后 (后续不再使用), 执行 `ShutdownSdk()`。

详细内容请参见: [简单上传](#)。

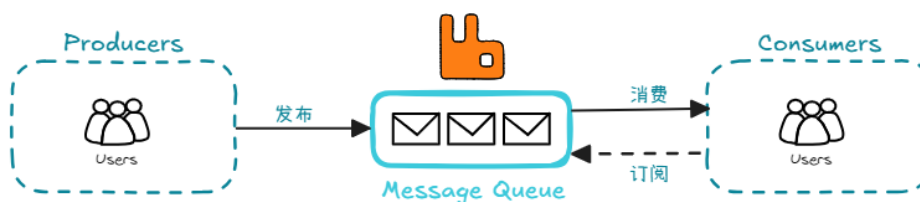
3.5 小结

第二期功能的实现，很好地展示了学习新技术的流程和方式：



4 第三期：消息队列 RabbitMQ

同步备份文件会导致响应时间变长，影响用户体验。我们可以改用消息队列实现异步备份，从而提升系统响应速度。



4.1 引入消息队列带来的好处

1. 异步解耦，提升响应速度

这是我们引入消息队列的核心原因。生产者（我们的应用程序）只需把备份任务作为“消息”丢进队列，立刻就能返回，无需等待耗时的备份操作完成。用户体验得到直接改善。

2. 削峰填谷，增强系统稳定性

在业务高峰期（如大促期间），瞬间的请求量可能压垮后端。消息队列能像大坝一样，把突发的流量先蓄起来（削峰），然后让后端服务按照自己能处理的速度去消费（填谷），避免后端被冲垮。

3. 故障隔离，提高系统可用性

当备份服务（消费者）临时宕机或网络抖动时，消息会安全地存储在队列中。待服务恢复后，它可以继续从断点处处理消息。生产者的核心业务不受影响，两者故障互相隔离。

4. 支持最终一致性

对于备份这类对实时一致性要求不高的场景（不需要备份立即完成），消息队列能很好地保证“最终一致性”——即只要没有意外，消息最终一定会被处理，备份最终一定会完成。

5. 弹性扩展

如果备份任务增多，可以方便地增加“消费者”（即更多的备份服务器）来处理队列中的消息，实现水平扩展。

4.2 RabbitMQ 简介

RabbitMQ 是一个广泛使用的开源消息队列中间件，它实现了 AMQP（高级消息队列协议），基于 Erlang 语言⁶ 开发，具备高可靠性、灵活的路由能力以及易于使用的管理界面。

它的核心设计理念是：生产者从不直接发送消息到队列，而是发送给“交换机”，由交换机根据提前配置好的路由规则，将消息分发到不同的队列中。

⁶Erlang：是一门专为高并发、高容错、分布式系统而生的函数式编程语言，由爱立信（Ericsson）公司于 1986 年设计。

4.3 使用 Docker 安装 RabbitMQ

拉取带管理插件的 *RabbitMQ* 镜像，包含 *Web* 可视化管理界面

```
docker pull rabbitmq:management
```

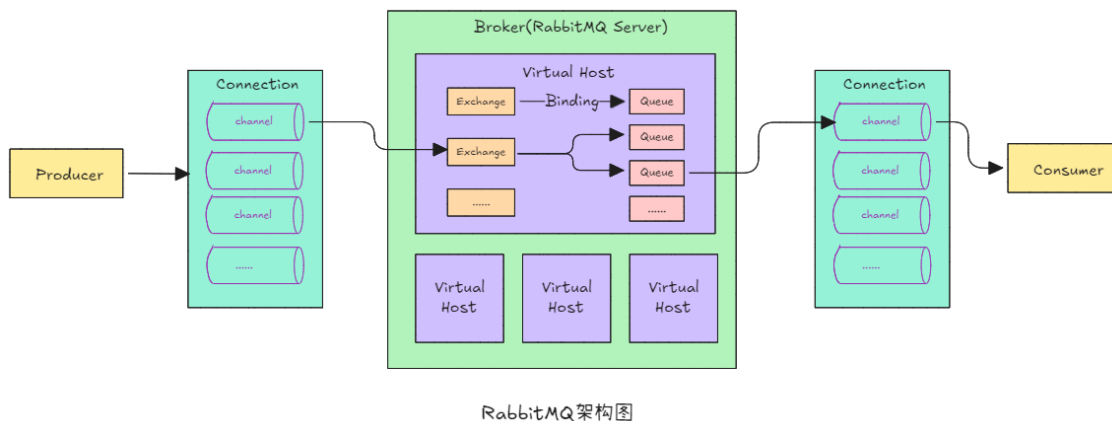
```
docker run -d \
--hostname rabbitsrv \
--name rabbit \
-p 5672:5672 \
-p 15672:15672 \
-p 25672:25672 \
-v /data/rabbitmq:/var/lib/rabbitmq \
rabbitmq:management
```

后台运行容器（守护模式）
设置容器内部主机名
容器名称
AMQP 协议端口（应用程序连接 *RabbitMQ* 使用）
管理界面端口（浏览器访问 <http://localhost:15672>）
集群通信端口
挂载数据目录（持久化消息队列数据）
使用带管理插件的 *RabbitMQ* 镜像

4.4 RabbitMQ 的架构和核心概念

这一小节，我们讲解 *RabbitMQ* 的架构和它的一些核心概念，这些是了解和使用 *RabbitMQ* 的前提。

RabbitMQ 的架构图如下所示：



RabbitMQ 的核心概念：

1. 生产者（*Producer*）— 发送消息的应用程序。
生产者创建消息并将消息发送到交换机，生产者可以指定消息的路由键（*Routing Key*），交换机会根据路由键判断消息应路由到哪个队列。
2. 消费者（*Consumer*）— 接收和处理消息的应用程序。
消费者从队列中接收消息，处理业务逻辑。消费者有两种消费模式：
 - 拉取模式（*Pull*）：消费者主动从队列拉取消息。
 - 推送模式（*Push*）：*RabbitMQ* 主动推送消息给消费者。
3. 队列（*Queue*）— 存储消息的容器，本质上是一个命名缓冲区。
4. 交换机（*Exchange*）— 接收生产者发送的消息，并根据路由规则将消息路由到一个或多个队列。
RabbitMQ 有四种类型的交换机：

类型	路由规则	作用
直连 (Direct)	消息的 <code>routingKey</code> 必须与队列绑定的 <code>bindingKey</code> 完全匹配。	单播
扇形 (Fanout)	忽略 <code>routingKey</code> ，广播给所有绑定的队列。	广播
主题 (Topic)	<code>routingKey</code> 支持通配符匹配： * 匹配一个字段， #: 匹配任意多个字段（包括 0 个）， 字段之间以 . 分隔。	根据多个条件路由
标头 (Headers)	根据消息的头属性匹配，忽略 <code>routingKey</code> 。	用于路由规则非常复杂的情况

5. 绑定 (*Binding*) — 连接交换机和队列的规则，定义了消息从交换机路由到队列的条件。

6. 路由键 (*Routing Key*) — 生产者在发送消息时指定的一个标签，用于交换机判断消息应路由到哪个队列。

7. 虚拟主机 (*Virtual Host, vhost*) — RabbitMQ 中的隔离环境，类似于 MySQL 的数据库。

虚拟主机的目的是实现多租户隔离，不同虚拟主机的交换机、队列、绑定是完全隔离的。默认的虚拟主机是 `/`。

8. 连接 (*Connection*) — 客户端和 RabbitMQ 服务器之间的 TCP 长连接。

一个连接可以包含多个通道 (*Channel*)，连接的创建和销毁开销比较大。

9. 通道 (*Channel*) — 在 *Connection* 内部创建的轻量级虚拟连接。

多个通道可以复用同一个连接（回顾：TCP 的多路复用）。每个通道都有自己独立的 ID。大部分 AMQP 协议的操作都可以在通道上完成。

这些核心概念是理解和使用 RabbitMQ 的基础，掌握它们就能应对 90% 以上的消息队列使用场景。

4.5 控制台操作

[[上课演示]]

4.6 安装 SimpleAmqpClient

```
# 更新软件包列表，获取最新的可用版本信息
sudo apt update
# SimpleAmqpClient 依赖 boost 库，安装 Boost 的全套开发文件（头文件及预编译库）
sudo apt install libboost-all-dev
# SimpleAmqpClient 依赖 rabbitmq-c，先安装 rabbitmq-c
cd rabbitmq-c-0.11.0/
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig
# 安装 SimpleAmqpClient
cd SimpleAmqpClient-2.5.1/
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig
```

安装成功后，/usr/local/lib/目录下会多三个库文件 libSimpleAmqpClient.so、libSimpleAmqpClient.so.7、libSimpleAmqpClient.so.7.0.1；/usr/local/include/目录下会多一个文件夹 SimpleAmqpClient，里面放的是头文件。

4.7 示例：生产者发送消息

```
----- rabbitmq_producer.cc -----
1 #include <SimpleAmqpClient/SimpleAmqpClient.h>
2 #include <string>
3
4 using namespace std;
5 using namespace AmqpClient;
6
7 int main()
8 {
9     // 1. 创建 Channel
10    string host = "127.0.0.1";
11    int port = 5672;
12    string username = "guest";
13    string password = "guest";
14    string vhost = "/";
15    Channel::ptr_t channel = Channel::Create(host, port, username, password, vhost);
16    // 2. 构建消息
17    BasicMessage::ptr_t message = BasicMessage::Create("Hello RabbitMQ");
18    // 3. 发送消息
19    string exchange = "oss.direct"; // 交换机
20    string routingKey = "oss"; // 消息的 routingKey
21    channel->BasicPublish(exchange, routingKey, message); // 发布消息
22 }
```

4.8 示例：消费者消费消息

```
----- rabbitmq_consumer.cc -----
1 #include <SimpleAmqpClient/SimpleAmqpClient.h>
2 #include <iostream>
3 #include <string>
4
5 using namespace std;
6 using namespace AmqpClient;
7
8 int main()
9 {
10    // 1. 以 URI 的方式创建 Channel
11    string uri = "amqp://guest:guest@localhost:5672/%2f";
12    Channel::ptr_t channel = Channel::CreateFromUri(uri);
13
14    // 2. 获取消息
15    // 方式 1: 拉取模式 --- 消费者主动从队列中拉取消息（非阻塞式）
16    // const string& q = "oss.queue";
17 }
```

```

18 // 如果队列中有消息，将消息放入到 envelope 中，并返回 true
19 // 如果队列中没有消息，BasicGet 会立刻返回 false
20 // Envelope::ptr_t envelope;
21 // channel->BasicGet(envelope, q);
22 // if (envelope && envelope->Message()) { */
23 //     cout << envelope->Message()->Body() << endl;
24 // }
25
26 // 方式 2: 推送模式 --- 等待 RabbitMQ 推送消息 (阻塞式)
27 const string& q = "oss.queue";
28 channel->BasicConsume(q); // 订阅队列
29 // 阻塞: 等待 RabbitMQ 推送消息
30 Envelope::ptr_t envelope = channel->BasicConsumeMessage();
31 // 打印消息
32 if (envelope && envelope->Message()) {
33     cout << envelope->Message()->Body() << endl;
34 }
35 }

```

5 第四期：微服务架构

前三期我们实现的服务器架构如下所示，它是一个单体应用⁷：

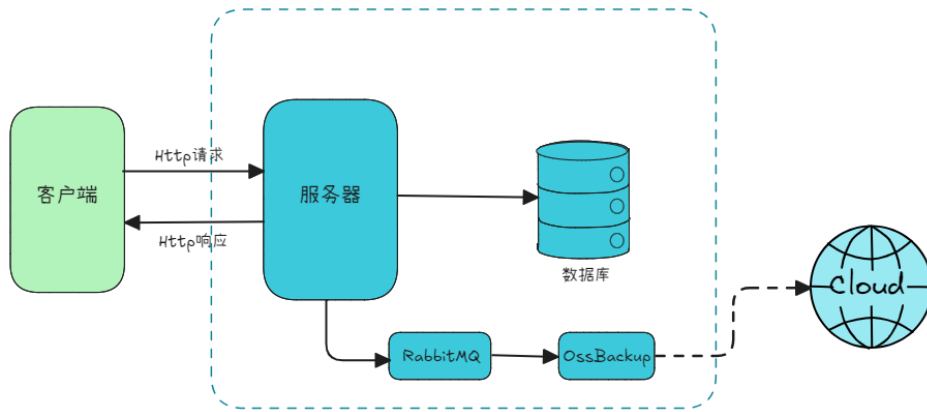


Figure 1: 单体应用

这一期，我们会将前面的单体应用改造成微服务应用⁸，以避免单体应用的一些缺点和局限性，如下图所示：

⁷ 单体应用：简单来说，就是把一个软件项目的功能模块（比如用户管理、订单处理、商品展示、数据库访问等）都打包成一个独立的、不可分割的整体来开发、部署和运行。

⁸ 微服务应用：就是一种将单个大型应用程序拆分为一组小型、独立服务的架构方式。

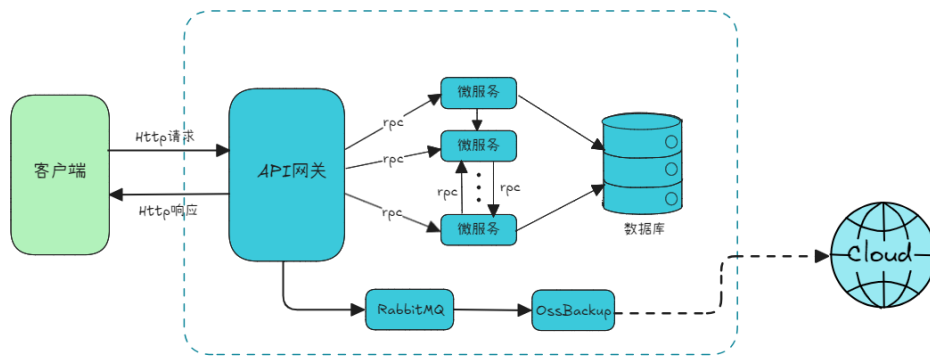


Figure 2: 微服务应用

5.1 单体应用 v.s. 微服务应用

单体应用

优点:

1. 部署简单: 只需要打包一个应用程序并部署到服务器即可。
2. 测试相对容易: 启动应用后, 就可以进行完整的集成测试, 因为所有组件都在一个进程中。
3. 性能表现可能更好: 模块间通过函数调用进行通信, 速度极快, 没有网络延迟和序列化的开销。

缺点:

1. 代码复杂度高, 维护困难: 随着功能增加, 代码库会变得巨大、臃肿且结构混乱, 难以理解和修改。
2. 技术栈僵化: 整个应用必须使用相同的技术栈 (语言、框架、库版本), 很难甚至无法为特定任务引入更合适的新技术。
3. 扩展性差: 即使只有一个功能 (如订单处理) 面临高并发, 你也必须复制整个应用程序的实例, 无法只扩展那个特定功能。这导致资源利用效率低下, 成本高昂。
4. 可靠性低: 任何一个模块中的一个微小 **bug** 都可能导致整个应用程序崩溃。
5. 交付和部署瓶颈: 牵一发而动全身: 任何微小的更改都需要重新构建和部署整个应用程序。随着团队规模扩大, 协同工作、代码合并和部署会变得非常缓慢和高风险, 无法实现敏捷交付。

微服务应用

优点:

1. 高可维护性与可理解性: 职责单一, 每个服务都很小, 只关注一个特定的业务功能, 代码更清晰, 更易于开发者理解和维护。
2. 独立部署: 服务可以独立开发、测试和部署。修复一个服务中的 **bug** 或发布一个新功能, 只需部署该服务本身, 无需重建和部署整个应用, 这大大加快了发布周期。
3. 技术选型灵活: 每个服务都可以使用最适合其需求的技术栈 (编程语言、数据库等)。例如, **AI** 服务可以用 **Python**, 高性能计算服务可以用 **C++**。
4. 可扩展性极强: 可以仅对那些需要更多资源的服务进行独立扩展。例如, 在电商促销时, 只扩展 “订单服务” 和 “支付服务”, 而 “用户评论服务” 保持不变。这极大地提高了资源利用率和成本效益。
5. 故障隔离: 一个服务的失败不会导致整个系统瘫痪。

缺点:

1. 分布式系统的复杂性。

2. 运维开销巨大。
3. 网络 and 性能开销。
4. 测试复杂度增加。

5.2 Protobuf

讲远程过程调用 (RPC) 之前, 我们先来讲讲 Protobuf。Protobuf 是一种数据交换格式, 它解决了 RPC 中最关键、最棘手的两个问题: 数据的“编解码”和“接口规范定义”。

简介

Protobuf (Google Protocol Buffer) 是一种轻便高效的数据交换格式, 它独立于语言和平台, 并且支持扩展。Protobuf 很适合做数据存储和 RPC 的数据交换格式。使用 Protobuf 内置的编译器 `protoc` 可以自动生成 C++, C#, Dart, Go, Java, Kotlin, Objective-C, Python, Ruby, PHP 等多种语言的代码。

安装

```
# 安装编译 Protobuf 所需的依赖工具
sudo apt-get install automake autoconf libtool
# 解压 Protobuf 源代码包 (tar: 归档文件, x: 解压, z: 通过 gzip 解压, v: 显示详细过程, f: 指定文件)
tar xzvf protobuf-3.20.1.tar.gz
# 进入解压后的 Protobuf 源代码目录
cd protobuf-3.20.1
# 运行自动生成脚本, 该脚本会生成 configure 配置文件
./autogen.sh
# 运行配置脚本: 检查系统环境、依赖项, 并生成 Makefile
./configure
# 编译源代码 (-j4 表示同时使用 4 个 CPU 核心并行编译, 加快速度)
make -j4
# 安装编译好的 Protobuf 到系统目录 (通常为 /usr/local/bin, /usr/local/lib 等)
sudo make install
# 更新系统的共享库缓存
sudo ldconfig
# 查看 Protobuf 编译器版本号
$ protoc --version
```

安装成功后, `/usr/local/include` 目录下会多一个 `google/protobuf` 文件夹, 里面放的是头文件。库文件则放在 `/usr/local/lib` 目录下。

使用教程

对 Protobuf 有了一定的基本了解之后, 接下来, 我们看看该如何使用 Protobuf。

1. 创建 `.proto` 文件, 定义数据结构, 如下所示:

```
syntax="proto3";
package test;
```

```
message Person {
    int32 id = 1;
    string name = 2;
    optional string email = 3;
}
```

我们在上面的例子中定义了一个名为 `Person` 的消息。定义消息的语法很简单，`message` 关键字后跟上消息的名称，之后在 `{}` 中定义 `message` 的字段，形式为：

```
message xxx {
    字段规则 类型 名称 = 字段编号;
    ...
}
// 字段规则:
//     required -> 字段只能也必须出现 1 次 (默认)
//     optional -> 字段可出现 0 次或 1 次
//     repeated -> 字段可出现任意多次 (包括 0)
// 类型: int32、int64、sint32、sint64、string、32-bit ...
```

2. 使用 `protoc` 编译 `.proto` 文件:

```
protoc --cpp_out=./ Person.proto
# Usage: protoc [OPTION] PROTO_FILES
# OPTIONS:
#     --cpp_out=OUT_DIR    指定在哪个目录下生成 C++ 的头文件和源文件
```

生成的头文件和源文件分别为: `Person.pb.h` 和 `Person.pb.cc`。这些文件中定义和实现了 `Person` 消息的各个方法，其中就有各个字段的 `getter` 和 `setter` 方法:

```
namespace test {

class Person final :
    ...
    int32_t id() const;
    void set_id(int32_t value);
    ...
    const std::string& name() const;
    template <typename ArgT0, typename... ArgT>
    void set_name(ArgT0&& arg0, ArgT... args);
    ...
    bool has_email() const;
    const std::string& email() const;
    template<typename ArgT0, typename... ArgT>
    void set_email(ArgT0&& arg0, ArgT... args);
    ...
};

}; // end of namespace `test`
```

3. 编写业务代码

这里我们演示一下如何使用 `Protobuf` 进行序列化和反序列化:


```

1 #include "Person.pb.h"
2 #include <fstream>
3 #include <iostream>
4 #include <string>
5
6 using namespace std;
7 using namespace test;
8
9 int main()
10 {
11     Person p1;
12     p1.set_name("test");
13     p1.set_id(100);
14     p1.set_email("example@gmail.com");
15
16     // 序列化: 将 C++ 中的结构体保存到二进制序列中
17     std::string output;
18     p1.SerializeToString(&output); // 序列化
19     cout << "size: " << output.size() << endl;
20     cout << "output: " << output << endl;
21
22     // 反序列化
23     Person p2;
24     cout << "p2.name: " << p2.name()
25           << ", p2.id: " << p2.id()
26           << ", p2.email: " << p2.email() << endl;
27
28     p2.ParseFromString(output); // 反序列化
29     cout << "p2.name: " << p2.name()
30           << ", p2.id: " << p2.id()
31           << ", p2.email: " << p2.email() << endl;
32 }

```

Protobuf 编码 *

我们通过一个具体的例子, 来看看 Protobuf 是如何编码的, .proto 文件还是采用上面的 Person.proto。

```

1 #include "Person.pb.h"
2 #include <bitset>
3 #include <iostream>
4
5 using namespace std;
6 using namespace test;
7
8 int main()
9 {
10     Person p;
11     p.set_id(100);
12     p.set_name("aaa");

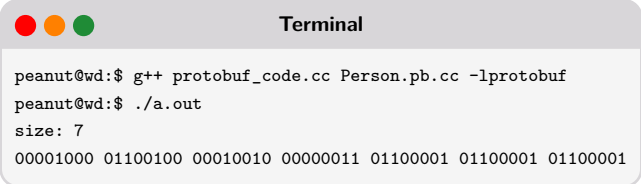
```

```

13
14     string data;
15     p.SerializeToString(&data); // 序列化
16
17     cout << "size: " << data.size() << endl;
18
19     for (auto c : data) {
20         cout << bitset<8>(c) << " ";
21     }
22     cout << endl;
23 }

```

运行结果如下：



```

Terminal
peanut@wd:$ g++ protobuf_code.cc Person.pb.cc -lprotobuf
peanut@wd:$ ./a.out
size: 7
00001000 01100100 00010010 00000011 01100001 01100001 01100001

```

解析：

```

00001:      序号：1
000:        数据编码格式 (WireType): Varint
01100100:   值：100
00010:      序号：2
010:        数据编码格式 (WireType): Length-delimited
00000011:   长度：3
01100001:   值：97('a')
01100001:   值：97('a')
01100001:   值：97('a')

```

Protobuf 的数据编码格式有 6 种，其中两种已标记为废弃：

Wire Type	名称	描述
0	Varint	可变长整数 (int32, int64, uint32, uint64, sint32, sint64, bool, enum)
1	64-bit	固定 64 位 (fixed64, sfixed64, double)
2	Length-delimited	长度前缀字段 (string, bytes, 嵌套 message, packed repeated fields)
3	Start group	group 的开始 (已废弃)
4	End group	group 的结束 (已废弃)
5	32-bit	固定 32 位 (fixed32, sfixed32, float)

5.3 sRPC

简介

RPC (*Remote Procedure Call*, 远程过程调用) 是一种计算机通信协议。它允许一个程序 (客户端) 像调用本地函数一样, 直接调用另一个地址空间 (通常是另一台机器上的程序) 中的函数或方法, 而无需显式编写网络通信的细节。

简单来说: 让调用远程服务像调用本地方法一样简单。而 **sRPC** 是由搜狗公司开发并开源, 基于 **Workflow** 异步引擎构建的一个 RPC 框架。

原理

理解 RPC（远程过程调用）的关键，在于理解它如何让“调用远程服务”变得像“调用本地函数”一样自然。

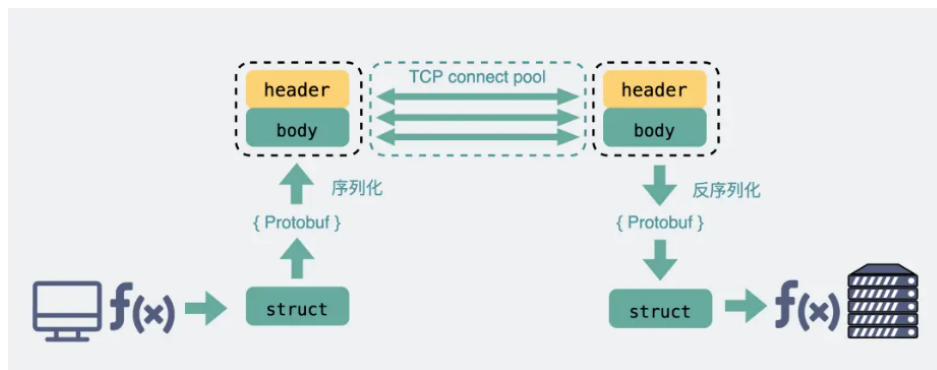


Figure 3: RPC 流程图

RPC 调用通常包含以下五个核心步骤：

1. 客户端调用远程方法 — `remoteAdd(3, 5)`
客户端代理拦截了这个调用。它负责把函数名 `remoteAdd` 和参数 3、5 按照某种约定的协议（如 Protobuf、JSON）序列化成一条二进制数据流。
2. 网络传输
客户端代理通过套接字把序列化后的数据传输到服务器。
3. 服务端接收与反序列化
服务端接收数据，并反序列化，从二进制数据中解析出函数名 `remoteAdd` 和参数 3、5。
4. 服务端调用函数并返回结果
服务器根据解析得到的函数名，调用相应的函数 `add`，得到结果 8。并把结果序列化，通过网络传给客户端。
5. 客户端接收结果
客户端接收服务器返回的二进制数据，并反序列化得到结果。

上面整个流程对程序员来说是完全透明的，就像在调用本地函数一样。

安装

```
# 安装依赖
sudo apt install liblz4-dev
sudo apt install libsnappy-dev
# 安装 srpc
tar xvfz srpc-0.10.2.tar.gz
cd srpc-0.10.2
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig
```

安装成功后，头文件放在 `/usr/local/include/srpc` 目录下，库文件则会安装到 `/usr/local/lib` 目录下，并且会生成一个可执行程序（生成代码的工具）`srpc_generator`。

使用教程

安装好 `srpc` 之后，接下来我们来看看该如何使用 `srpc`:

1. 编写 IDL (Interface Definition Language) 文件

```
example.proto
1 syntax = "proto3"; // proto2 和 proto3 都可以，推荐使用 proto3
2
3 // 定义请求消息
4 message EchoRequest {
5     string message = 1;
6     string name = 2;
7 };
8 // 定义响应消息
9 message EchoResponse {
10     string message = 1;
11 }
12
13 // 定义一个名字为 Example 的服务
14 service Example {
15     // 定义一个名字叫 Echo 的 rpc
16     rpc Echo(EchoRequest) returns (EchoResponse);
17     // 一个 service (服务) 中，可以定义多个 rpc 方法...
18 }
```

2. 使用 `protoc` 和 `srpc_generator` 生成代码

```
protoc --cpp_out=./ example.proto
```

上面命令会生成两个文件，分别为: `example.pb.h` 和 `example.pb.cc`，里面包含 IDL 文件中 `message` 的定义和实现。

```
srpc_generator protobuf example.proto ./
# Usage:
#     srpc_generator [protobuf/thrift] <idl_file> <output_dir>
```

上面命令会生成三个文件，分别为: `client.pb_skeleton.cc`, `server.pb_skeleton.cc`, `example.srpc.h`。其中 `client.pb_skeleton.cc` 和 `server.pb_skeleton.cc` 分别为客户端和服务端的骨架代码（只提供骨架，还需要程序员修改）。`example.srpc.h` 包含了 IDL 文件中 `service` 内容的定义：

```
...
namespace Example
{

class Service : public srpc::RPCService
{
public:
    // 纯虚函数：需要派生类重写（RPC 服务器实现）
    virtual void Echo(EchoRequest *request, EchoResponse *response,
        srpc::RPCContext *ctx) = 0;
public:
    Service();
}
```

```

};

using EchoDone = std::function<void (EchoResponse *, srpc::RPCContext *)>;

class SRPCClient : public srpc::SRPCClient
{
public:
    void Echo(const EchoRequest *req, EchoDone done);    // 异步
    void Echo(const EchoRequest *req, EchoResponse *resp, srpc::RPCSyncContext *sync_ctx);    // 同步
    WFFuture<std::pair<EchoResponse, srpc::RPCSyncContext>> async_Echo(const EchoRequest *req);
public:
    SRPCClient(const char *host, unsigned short port);
    SRPCClient(const struct srpc::RPCClientParams *params);
public:
    srpc::SRPCClientTask *create_Echo_task(EchoDone done);    // 与 workflow 集成用的
};

}    // end of namespace `Example`

```

3. 修改客户端和服务端的骨架代码，实现具体的业务逻辑。
详细代码见：examples/srpc/目录

需要链接的库

链接时，需要加上-lsrpc -llz4 -lsnappy -lprotobuf -lworkflow

6 第五期：注册中心 Consul

在上一期中，我们将登录注册逻辑从 API 网关中解耦出来。然而，API 网关在调用登录注册服务时，仍然采用硬编码的 IP 地址和端口，这意味着网关与后端服务之间并未实现彻底解耦。一旦后端服务发生宕机或地址变更，API 网关将无法正常工作。为了解决这一问题，我们可以同时启动多个登录注册服务的实例，并在启动时，将自己的网络地址等信息注册到注册中心。服务的消费者从注册中心查询可用的地址，这样就无需硬编码 IP 地址和端口了。

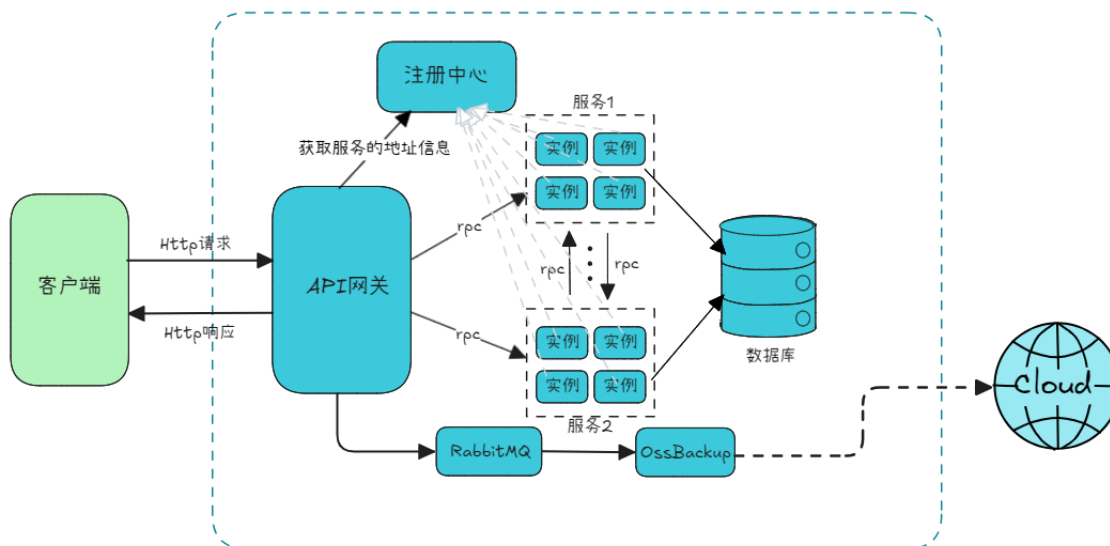


Figure 4: 引入注册中心的微服务架构

整个流程如下：

1. 实例启动时，向注册中心注册自己的元数据（如：服务的名字、IP 地址、端口等信息）。
2. 注册中心会定期检查各个实例的健康状态，并更新各个实例的健康信息（可用不可用）。
3. 服务的消费者从注册中心获取服务的某个健康实例的地址信息。
4. 服务的消费者根据从注册中心获取的地址信息，发送 RPC 请求。

6.1 简介

注册中心是微服务架构中的核心组件之一，它的主要作用是管理和维护各个微服务的注册信息，实现服务之间的自动注册和发现。注册中心 (Service Registry) 是一个用于保存服务实例信息的数据库或服务。每个微服务在启动时，会将自己的地址 (如 IP 地址和端口) 和相关元信息注册到注册中心；其他服务可以通过注册中心找到它需要调用的服务。

6.2 使用 Docker 安装 Consul

启动第一个节点

```
docker run --name consul1 -d -p 8500:8500 -p 8301:8301 -p 8302:8302 -p 8600:8600 \
hashicorp/consul agent -server -bootstrap-expect 2 -ui -bind=0.0.0.0 -client=0.0.0.0
```

查看第一个节点的 IP 地址，本例子中是：172.17.0.3

```
docker inspect consul1
```

加入第二个节点，注意：-join 后面的 ip 地址应与上面查询的结果一致

```
docker run --name consul2 -d -p 8501:8500 hashicorp/consul agent -server -ui \
-bind=0.0.0.0 -client=0.0.0.0 -join 172.17.0.3
```

加入第二个节点，注意：-join 后面的 ip 地址应与上面查询的结果一致

```
docker run --name consul3 -d -p 8502:8500 hashicorp/consul agent -server -ui \
-bind=0.0.0.0 -client=0.0.0.0 -join 172.17.0.3
```

6.3 安装 ppconsul

ppconsul 是一个用 C++11 编写的 Consul 客户端库，旨在为 C++ 开发者提供简单、模块化且高效的 API，用于与 Consul 进行交互。它将 Consul 的 HTTP API 封装成 C++ 风格、类型安全的接口，让 C++ 微服务也能轻松接入 Consul 进行服务发现和配置管理。

ppconsul 是一个典型的 CMake 工程，依照其它 CMake 的安装流程即可。

```
cd ppconsul-0.2.3
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig
```

安装成功后，头文件放置在/usr/local/include/ppconsul 目录下，库文件位于/usr/local/lib 目录。

6.4 使用教程

```
consul_client.cc
1 #include "common.h"
2 #include <iostream>
3 #include <ppconsul/agent.h>
4 #include <wfrest/HttpServer.h>
5 #include <workflow/WFFacilities.h>
6
7 using namespace std;
8 using namespace wfrest;
9 using ppconsul::Consul;
10 using namespace ppconsul::agent;
11
12 WFFacilities::WaitGroup waitGroup(1);
13
14 void sig_handler(int)
15 {
16     waitGroup.done();
17 }
18
19 void timer_callback(WFTimerTask* task)
20 {
21     int state = task->get_state();
22     if (state != WFT_STATE_SUCCESS) {
23         return;
24     }
25     SeriesWork* series = series_of(task);
26     Agent* agent = static_cast<Agent*>(series->get_context());
27
28     // 发送心跳包
29     agent->servicePass("UserService1");
30     // 创建另一个定时任务
31     WFTimerTask* nextTask = WFTaskFactory::create_timer_task("health_check", 9, 0, timer_callback);
```

```

32     series->push_back(nextTask);
33 }
34
35 int main()
36 {
37     HttpServer server;
38
39     if (server.start(8888) == 0) {
40
41         // 指定注册中心 Consul 的 ip 地址, 端口和数据中心
42         Consul consul("http://127.0.0.1:8500", ppconsul::kw::dc = "dc1");
43         // 创建 Consul 客户端代理
44         Agent agent(consul);
45         // 注册实例的元信息
46         agent.registerService(
47             kw::id = "UserService1",
48             kw::name = "UserService",
49             kw::address = "127.0.0.1",
50             kw::port = 8888,
51             kw::check = TtlCheck { chrono::seconds(10) });
52
53         // 定时发送心跳包
54         WFTimerTask* timerTask = WFTaskFactory::create_timer_task("health_check", 9, 0, timer_callback);
55
56         SeriesWork* series = Workflow::create_series_work(timerTask, nullptr);
57         series->set_context(&agent); // 设置序列的上下文
58         series->start();
59
60         waitGroup.wait();
61         server.stop();
62     } else {
63         cerr << "Error: Server start FAILED!" << endl;
64         exit(1);
65     }
66     return 0;
67 }

```

服务注册好之后, 服务的消费者可以通过 URL:

`http://<consul_host>:8500/v1/health/service/<service_name>?passing=true`

查找某个服务的健康实例。Consul 服务器会返回类似下面的 JSON 数据:

```

[
{
    "Node": {
        ...
    },
    "Service": {
        "ID": "UserService1",
        "Service": "UserService",

```



```
"Tags": [],  
"Address": "127.0.0.1",  
"TaggedAddresses": {  
  "lan_ipv4": {  
    "Address": "127.0.0.1",  
    "Port": 8888  
  },  
  "wan_ipv4": {  
    "Address": "127.0.0.1",  
    "Port": 8888  
  }  
},  
"Meta": {  
},  
"Port": 8888,  
...  
}  
}  
]
```

解析 JSON，我们就能得到实例的 IP 地址和端口信息。